DOCKER ADMINISTRATION - TÀI LIỆU ÔN TẬP

Dành cho người mới bắt đầu - bám theo lab self-host Mascot trên Oracle Linux

Mục tiêu của tài liệu này là giúp bạn hiểu Docker theo bản chất, nhưng bằng ngôn ngữ rất đơn giản: Docker là gì, image/container khác nhau ra sao, network/port/volume hoạt động thế nào, Compose làm gì, cách đọc logs/health và cách suy luận khi hệ thống lỗi.

Cách học: Đừng học thuộc lệnh. Với mỗi lệnh, hãy tự hỏi: mình đang muốn kiểm tra layer nào, output nào chứng minh nó hoạt động, và nếu sai thì bước tiếp theo là gì?

I. Bức tranh tổng thể: Docker nằm ở đâu trong Mascot?

Trong hệ thống self-host Mascot, Oracle Linux là máy chủ. PostgreSQL 18 chạy trực tiếp trên Linux host. Frontend và backend chạy trong Docker container.

```
Windows
 -> VirtualBox
   -> Oracle Linux 9.7
      |- PostgreSQL 18 :5432
      |- Docker Engine
      |   |- Backend container :8000
      |   `- Frontend container :5173
      `- firewalld / SSH
```

Điểm quan trọng: container không phải là một máy ảo đầy đủ. Container dùng chung Linux kernel của Oracle Linux nhưng có môi trường process, network và filesystem được cách ly.

1. Docker khác VM thế nào?

```
VM: Host OS -> Hypervisor -> Guest OS -> App
Docker: Host Linux -> Docker -> Container -> App
```

- VM có guest OS/kernel riêng.
- Container dùng chung kernel của host Linux.
- Vì không cần một guest OS đầy đủ cho mỗi container, container thường nhẹ và khởi động nhanh hơn VM.

II. Docker Architecture: ai thực sự chạy container?

```
docker command
 -> Docker CLI
   -> Docker daemon (dockerd)
     -> containerd
       -> container process
```

Khi bạn gõ docker ps, CLI chỉ gửi yêu cầu. Docker daemon mới là service chạy nền và quản lý image, container, network, volume.

Các lệnh cần nhớ

```
docker version
sudo systemctl status docker
```

docker version cho biết client/server Docker. systemctl status docker kiểm tra Docker daemon trên Linux có đang chạy hay không.

Bản chất: Docker CLI tồn tại không có nghĩa Docker daemon đang chạy. Nếu daemon chết, bạn có thể thấy lỗi Cannot connect to the Docker daemon.

III. Dockerfile -> Image -> Container

```
Dockerfile
  --docker build--> Image
  --docker run----> Container
```

1. Dockerfile

Dockerfile là công thức mô tả cách đóng gói ứng dụng thành image.

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install ...
CMD ["uvicorn", "..."]
```

- FROM: chọn base image.
- WORKDIR: thư mục làm việc bên trong image/container.
- COPY: đưa source/file vào image.
- RUN: chạy lệnh trong quá trình build image.
- CMD: command mặc định khi container start.

2. Image

Image là “bản đóng gói” của ứng dụng. Nó chứa filesystem, runtime, dependency và metadata để tạo container. Image không phải process đang chạy.

3. Container

Container là một instance được tạo từ image. Một image có thể tạo nhiều container.

```
backend image
  |- backend-1
  |- backend-2
  `- backend-3
```

4. Lệnh quan sát

```
docker images
docker ps
docker ps -a
```

- docker images: xem image.
- docker ps: xem container đang chạy.
- docker ps -a: xem cả container running, stopped, exited...

IV. Image Layer và Build Cache

Docker image được xây từ nhiều layer. Mỗi instruction quan trọng trong Dockerfile có thể tạo ra một layer hoặc ảnh hưởng tới cache.

```
Layer 1: base Python
Layer 2: dependency metadata
Layer 3: installed dependencies
Layer 4: application source
```

Nếu dependency không đổi nhưng source đổi, Docker có thể tái sử dụng layer dependency đã cache. Đây là lý do build sau có thể nhanh hơn.

```
docker history mascot-backend
```

V. Container Lifecycle và docker inspect

Container có nhiều trạng thái: created, running, exited, restarting, paused, dead. Nếu có healthcheck còn có starting, healthy, unhealthy.

```
docker start <container>
docker stop <container>
docker restart <container>
docker rm <container>
docker inspect <container>
```

docker inspect cho metadata chi tiết: image, network, IP, port mapping, environment, mounts, restart policy, healthcheck...

Một số lệnh inspect gọn

```
docker inspect -f '{{.Config.Image}}' mascot-backend-1
docker inspect -f '{{.HostConfig.RestartPolicy.Name}}' mascot-backend-1
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' mascot-backend-1
```

Exited (1) nghĩa là gì?

Nó thường có nghĩa process chính trong container đã thoát với exit code 1. Không được vội kết luận Docker Engine hỏng.

```
container start
 -> main process runs
```

```
-> process crashes
-> exit code 1
-> container becomes Exited (1)
```

Ví dụ log ModuleNotFoundError thường là lỗi application/dependency: package thiếu, image cũ chưa rebuild, hoặc dependency chưa được khai báo đúng.

VI. Running khác Healthy - và vì sao logs quan trọng

Running chỉ chứng minh process chính của container còn sống. Nó không chứng minh application bên trong hoạt động đúng.

```
running != healthy
```

Backend có thể vẫn "Up" nhưng database password sai, DB down hoặc endpoint lỗi. Vì vậy healthcheck và logs rất quan trọng.

Logs

```
docker logs mascot-backend-1
docker compose -f docker-compose.selfhost.yml logs backend
docker compose -f docker-compose.selfhost.yml logs --tail=50 backend
docker compose -f docker-compose.selfhost.yml logs -f backend
```

Khi API trả 500, bước đầu nên xem application logs trước. docker inspect hữu ích khi nghi cấu hình container; logs hữu ích khi application đang báo lỗi.

Quy tắc: App báo lỗi -> xem logs. Nghi container config/network/mount/restart policy -> inspect.

Ví dụ suy luận lỗi password

```
password authentication failed for user "mascot_user"
```

Lỗi này rất có giá trị vì nó chứng minh backend đã đi tới PostgreSQL server. DNS/hostname, routing, TCP và PostgreSQL process/port đã hoạt động đủ để DB trả lỗi authentication.

```
Backend -> network -> PostgreSQL :5432  OK
PostgreSQL checks password          FAIL
```

VII. Docker Network - phần quan trọng nhất với Mascot

```
docker network ls
docker network inspect mascot_default
```

Docker Compose thường tạo một bridge network riêng cho project, ví dụ mascot_default. Backend và frontend được gắn vào network này và nhận private IP kiểu 172.18.x.x.

```
Oracle Linux
 |- enp0s3: 192.168.x.x    <- IP LAN của VM
 |- docker0: 172.17.0.1    <- Docker bridge
```

```
`- br-xxxx: 172.18.0.1     <- Compose bridge
   |- backend: 172.18.0.2
   `- frontend: 172.18.0.3
```

IP 192.168.x.x được Windows dùng để truy cập VM. IP 172.18.x.x chủ yếu dành cho mạng nội bộ Docker.

Service name và DNS nội bộ

Các container trong cùng Compose network có thể resolve nhau bằng service name. Không nên hardcode IP container vì container có thể recreate và nhận IP mới.

localhost bên trong container

Đây là chỗ rất dễ nhầm: localhost luôn chỉ chính network namespace hiện tại.

```
Trên Oracle Linux: localhost = Oracle Linux
Trong backend container: localhost = backend container
```

Vì PostgreSQL Mascot chạy trên Linux host, backend container không thể dùng localhost:5432 để tìm PostgreSQL host.

host.docker.internal / host-gateway

```
Backend container
 -> host.docker.internal
 -> Docker host gateway
 -> Oracle Linux
 -> PostgreSQL :5432
```

Trong Compose, mapping kiểu host.docker.internal:host-gateway giúp container có hostname ổn định để đi tới host.

VIII. Port Mapping: HOST_PORT : CONTAINER_PORT

```
ports:
  - "8000:8000"
```

Cách đọc luôn là HOST : CONTAINER.

```
Windows -> VM-IP:8000
        -> Oracle Linux host port 8000
        -> Docker publish/NAT
        -> backend container port 8000
```

Nếu cấu hình 8080:8000 thì browser phải gọi VM-IP:8080; Docker mới forward vào port 8000 của container.

```
docker ps
docker port mascot-backend-1
sudo ss -lntp | grep 8000
```

Nếu docker ps hiển thị 0.0.0.0:8000->8000/tcp, host publish port 8000 trên mọi IPv4 interface. Nếu chỉ bind 127.0.0.1:8000, máy ngoài VM thường không truy cập được.

IX. Docker port đã publish vẫn có thể bị firewall chặn

Đường đi từ Windows tới backend còn phải qua network interface và firewalld của Oracle Linux.

```
Windows
 -> VM LAN IP
 -> firewalld
 -> host port 8000
 -> Docker
 -> backend container

sudo firewall-cmd --list-ports
```

Trong lab Mascot, frontend/backend cần 5173/tcp và 8000/tcp. PostgreSQL 5432 không cần mở ra LAN nếu chỉ backend container cần truy cập DB.

X. Volume và Persistence

Writable filesystem bên trong container không nên được coi là nơi lưu dữ liệu lâu dài. Container có thể bị remove/recreate.

```
Container
 -> writable layer (temporary-ish)

Persistent data
 -> Docker volume hoặc bind mount
```

1. Volume

Volume là storage do Docker quản lý. Nó tồn tại độc lập hơn với lifecycle container.

```
docker volume ls
docker volume inspect <volume>
```

2. Bind mount

Bind mount lấy một path thật trên host rồi mount vào container, ví dụ ./src:/app/src.

3. Mascot hiện tại

PostgreSQL 18 chạy trực tiếp trên Oracle Linux host, nên database data không phụ thuộc Docker volume. Đây là khác biệt với kiến trúc PostgreSQL chạy trong container.

XI. Docker Compose

Docker Compose cho phép mô tả nhiều container của một application trong một file YAML thay vì chạy hàng loạt docker run dài.

```
services:
build:
image:
ports:
environment:
```

```
volumes:
depends_on:
healthcheck:
restart:
networks:
```

Các lệnh quan trọng

```
docker compose -f docker-compose.selfhost.yml up -d
docker compose -f docker-compose.selfhost.yml down
docker compose -f docker-compose.selfhost.yml ps
docker compose -f docker-compose.selfhost.yml build
docker compose -f docker-compose.selfhost.yml logs
```

- up -d: tạo/start các service ở detached mode.
- down: stop và remove container/network thuộc Compose project.
- ps: xem trạng thái service/container.
- build: build image từ Dockerfile.
- logs: xem log các service.

build khác up

```
docker compose build
 -> Dockerfile -> image

docker compose up
 -> đọc Compose -> network -> container -> start
```

restart: unless-stopped

Policy này giúp container tự khởi động lại khi Docker daemon/server reboot, trừ khi người quản trị đã chủ động stop container. Đây là lý do Mascot có thể tự lên lại sau reboot.

depends_on không đồng nghĩa application đã ready

depends_on chủ yếu mô tả dependency/start order. Container started chưa chắc application ready. Healthcheck mới giúp kiểm tra mức sẵn sàng theo tiêu chí cụ thể.

XII. Environment Variables và vì sao sửa .env có thể chưa đủ

```
.env trên host
 -> Docker Compose đọc
 -> inject environment vào container
 -> application đọc
```

Container đang chạy không tự thay đổi environment khi file .env trên host thay đổi. Vì vậy khi config thay đổi, thường phải recreate container.

```
docker compose -f docker-compose.selfhost.yml up -d --force-recreate
```

Đây chính là dạng vấn đề có thể xảy ra khi VM đổi IP nhưng frontend API URL/CORS vẫn dùng IP cũ.

Bảo mật: Không public toàn bộ docker inspect hoặc printenv nếu container có secret như DB password, JWT secret, Groq/Resend key.

XIII. Resource Monitoring và Disk

```
docker stats
```

docker stats cho biết CPU, RAM, network I/O, block I/O và số process của container. Nó cho biết triệu chứng tài nguyên, chưa tự nói nguyên nhân.

```
docker system df
```

docker system df giúp xem image, container, volume, build cache đang chiếm disk bao nhiêu.

docker system prune có thể xóa tài nguyên không dùng, nhưng không nên chạy bừa trên server. Trước tiên phải hiểu thứ gì sẽ bị xóa và liệu có cần rollback/debug hay không.

XIV. Troubleshooting theo Layer - kỹ năng quan trọng nhất

Đừng debug bằng cách chạy command ngẫu nhiên. Hãy xác định request đang đi đến đâu và dừng ở layer nào.

```
Browser
 -> VM IP/network
 -> firewall
 -> Docker published port
 -> container
 -> application
 -> database
```

Case A - backend Exited (1)

```
docker compose ps
 -> backend Exited (1)
 -> docker logs / compose logs
 -> đọc exception
 -> fix root cause
 -> rebuild/recreate nếu cần
```

Không restart bừa trước khi đọc log.

Case B - backend Up nhưng API trả 500

Up chỉ chứng minh process sống. 500 cho biết request đã tới application nhưng application xử lý lỗi. Ưu tiên xem backend logs.

Case C - localhost:8000 trên VM OK nhưng Windows không vào được

Lúc này backend/container/DB có khả năng đều ổn. Hãy chuyển focus sang đường Windows -> VM: IP LAN, Docker port publish, host listen, firewall, mạng công ty.

```
ip a / hostname -I
docker ps / docker port
ss -lntp
firewall-cmd --list-ports
```

Case D - connection refused tới DB

Có thể hostname sai, service DB không listen, port sai, routing/firewall sai, hoặc backend đang dùng localhost sai ngữ cảnh.

Case E - no pg_hba.conf entry

Lỗi này chứng minh connection đã tới PostgreSQL. Layer lỗi là access/authentication rule của PostgreSQL, không phải Docker Engine.

Case F - Cannot connect to Docker daemon

```
sudo systemctl status docker
sudo journalctl -u docker
```

Đây là layer Docker daemon/service.

Case G - permission denied Docker socket

Nếu sudo docker ps chạy nhưng docker ps không chạy, Docker daemon vẫn sống. Đây là Linux permission/group problem. User thường cần group docker và logout/login để session nhận group mới.

Case H - .env đổi nhưng app vẫn config cũ

Container hiện tại vẫn giữ environment cũ. Recreate container để inject config mới.

XV. Những câu phải tự giải thích được sau khi ôn

- Docker container có phải VM không? Vì sao?
- Docker CLI và Docker daemon khác nhau thế nào?
- Dockerfile, image và container khác nhau ở đâu?
- docker ps và docker ps -a khác nhau thế nào?
- Running và healthy khác nhau thế nào?
- Exited (1) nói điều gì về process chính?
- 8000:8000 được đọc theo thứ tự nào?
- IP 172.18.x.x khác IP 192.168.x.x ra sao?
- Tại sao localhost trong container không phải Linux host?
- host.docker.internal giải quyết bài toán gì trong Mascot?
- Volume và bind mount dùng khi nào?
- Compose build và up khác nhau thế nào?
- Tại sao sửa .env có thể cần recreate?
- docker logs, docker inspect và docker stats dùng cho ba loại câu hỏi khác nhau thế nào?
- Khi app lỗi, vì sao cần debug theo layer thay vì restart tất cả?

XVI. Cheat Sheet - lệnh cần nhớ theo mục đích

Mục đích	Lệnh
Docker service	sudo systemctl status docker
Image	docker images / docker history IMAGE
Container	docker ps / docker ps -a
Chi tiết container	docker inspect CONTAINER
Log	docker logs CONTAINER / docker compose ... logs backend
Network	docker network ls / docker network inspect NETWORK
Port	docker port CONTAINER / ss -lntp
Volume	docker volume ls / docker volume inspect VOLUME
Resource	docker stats
Disk	docker system df
Compose status	docker compose -f docker-compose.selfhost.yml ps
Start	docker compose -f docker-compose.selfhost.yml up -d
Build	docker compose -f docker-compose.selfhost.yml build
Recreate config	docker compose -f docker-compose.selfhost.yml up -d --force-recreate

XVII. Flow ôn tập 30 phút sau này

1. Vẽ: Dockerfile -> Image -> Container
2. Vẽ: Windows -> VM -> Firewall -> Docker port -> Backend -> PostgreSQL
3. Chạy: docker ps, logs, network inspect, docker stats
4. Tự giải thích localhost / host.docker.internal / port mapping
5. Làm 3 case: Exited(1), Up nhưng 500, VM OK nhưng Windows không vào được

Nếu bạn làm được 5 bước này mà không nhìn tài liệu, bạn đã nắm được nền tảng Docker Administration cần thiết cho task Mascot và đủ cơ sở để đi sâu hơn vào production Docker sau này.